

```

"""
SISTEMA LEGADO - XYZ
Aviso aos engenheiros: Este código está acoplado e difícil de manter!
Sua missão é refatorar utilizando os padrões Composite e Decorator.
"""

# =====
# PROBLEMA 1: A EXPLOÇÃO DE SUBCLASSES (Herança Rígida)
# =====
class MaquinaVirtual:
    def get_descricao(self):
        return "VM Simples"

    def get_preco(self):
        return 100.0

class BancoDeDados:
    def get_descricao(self):
        return "Banco de Dados SQL"

    def get_preco(self):
        return 250.0

# O pesadelo começa aqui... Para cada serviço extra, criaram uma nova subclasse!
class MaquinaVirtualComBackup(MaquinaVirtual):
    def get_descricao(self):
        return super().get_descricao() + " + Backup"

    def get_preco(self):
        return super().get_preco() + 50.0

class MaquinaVirtualComFirewall(MaquinaVirtual):
    def get_descricao(self):
        return super().get_descricao() + " + Firewall"

    def get_preco(self):
        return super().get_preco() + 80.0

# E se o cliente quiser os DOIS serviços? A herança múltipla ou classes gigantes começam a aparecer.
class MaquinaVirtualComBackupEFirewall(MaquinaVirtual):
    def get_descricao(self):
        return super().get_descricao() + " + Backup + Firewall"

    def get_preco(self):
        return super().get_preco() + 50.0 + 80.0

# Imagine criar classes assim para o Banco de Dados também...

# =====
# PROBLEMA 2: ACOPLAMENTO E FALTA DE UNIFORMIDADE
# =====
class Cluster:
    def __init__(self, nome):
        self.nome = nome
        self.itens = [] # Mistura de VMs, Bancos e suas variações

    def adicionar_item(self, item):
        self.itens.append(item)

    def calcular_fatura_cluster(self):
        total = 0
        # O código precisa testar o tipo de cada objeto manualmente (Violação do Open/Closed)
        for item in self.itens:

```

```

        if isinstance(item, MaquinaVirtual) or \
            isinstance(item, MaquinaVirtualComBackup) or \
            isinstance(item, MaquinaVirtualComFirewall) or \
            isinstance(item, MaquinaVirtualComBackupEFirewall):
            total += item.get_preco()
        elif isinstance(item, BancoDeDados):
            total += item.get_preco()
        else:
            print(f"Erro: Tipo de recurso não reconhecido no cluster {self.nome}")
    return total

class DataCenter:
    def __init__(self, nome):
        self.nome = nome
        self.clusters = []
        self.tem_suporte_vip = False # Regra de negócio hardcoded (engessada)

    def adicionar_cluster(self, cluster):
        self.clusters.append(cluster)

    def habilitar_suporte_vip(self):
        self.tem_suporte_vip = True

    def calcular_fatura_global(self):
        total = 0
        for cluster in self.clusters:
            # DataCenter chama um método diferente do Cluster (Falta de uniformidade)
            total += cluster.calcular_fatura_cluster()

        # O Suporte 24x7 está travado dentro do DataCenter.
        # E se o cliente quiser suporte só em um Cluster específico? É impossível na arquitetura
        atual!
        if self.tem_suporte_vip:
            total = total * 1.10 # Adiciona 10%

        return total

# =====
# CÓDIGO CLIENTE (Execução do Cenário)
# =====
if __name__ == "__main__":
    print("--- FATURAMENTO MENSAL NEBULA CLOUD (SISTEMA LEGADO) ---\n")

    # Montando Cluster Alpha
    cluster_alpha = Cluster("Cluster Alpha")
    cluster_alpha.adicionar_item(MaquinaVirtual())
    cluster_alpha.adicionar_item(MaquinaVirtual())
    cluster_alpha.adicionar_item(BancoDeDados())

    # Montando Cluster Beta
    cluster_beta = Cluster("Cluster Beta")
    # Instanciando a classe monstruosa
    vm_segura = MaquinaVirtualComBackupEFirewall()
    cluster_beta.adicionar_item(vm_segura)

    # Montando Data Center Principal
    datacenter = DataCenter("DataCenter SP")
    datacenter.adicionar_cluster(cluster_alpha)
    datacenter.adicionar_cluster(cluster_beta)

    # Habilitando o serviço de suporte engessado
    datacenter.habilitar_suporte_vip()

    # Impressão do resultado

```

```
# Perceba que não conseguimos imprimir a descrição da árvore inteira facilmente
print(f"Total a pagar pelo {datacenter.nome}: R$ {datacenter.calcular_fatura_global():.2f}")

print("\n[!] AVISO DA DIRETORIA:")
print("Precisamos lançar os serviços 'Anti-DDoS' e 'IP Dedicado'.")
print("O time de engenharia informou que precisará criar 16 novas subclasses")
print("e alterar dezenas de 'ifs' para isso. Salvem nosso sistema!")
```